Automating Clinical Trial Template Creation for Compliance and Efficiency

ENGR 250 Team 7
Vikram Arunkumar, Suriya Chellappan, Rahhul Jayaprakash

Problem

- Clinical trials require highly structured, compliant, and auditable documents
- Manual authoring is slow, costly, and prone to regulatory risk
- Frequent updates needed as protocols and regulations change

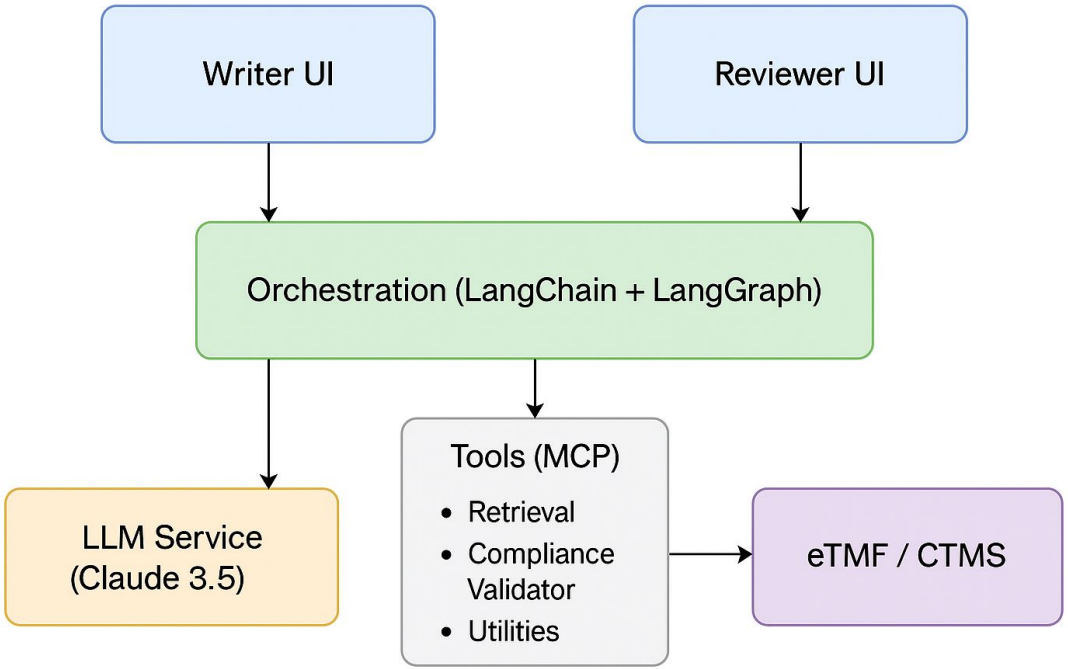
Key Challenges in Clinical Documentation

- Managing context across trial phases, amendments, and team input
- Ensuring compliance with constantly updated regulatory requirements
- Integrating latest medical literature and data in real time
- Maintaining transparent audit trails for all changes

Our AI-driven Solution for Trials

Clinical Documentation Challenge	Our Solution
Protocol complexity, evolving context	AI agent orchestration (LangChain)
Audit trail and compliance	Formal workflow with full logging (LangGraph)
Incorporating latest science and rules	Integrated real-time retrieval
Many revisions, team feedback	Automated iterative drafting and validation (Claude)

High Level Architecture



LangChain + Anthropic Claude 3.5 Sonnet

- Claude integrates with search engines (e.g., Brave Search)
- Real-time info retrieval for queries beyond training cutoff
- Pipeline: Query rewrite → Retrieval → Filtering → Synthesis
- Develop system to generate clinical trial document templates
- Improve speed, accuracy, and compliance in documentation workflows

Langchain

LangChain is an open-source framework that helps developers build applications powered by Large Language Models (LLMs) that can reason, use tools, and interact with external systems.

It provides a structured way to:

1. Connect LLMs (like Claude, GPT, Gemini, etc.)
2. Use external tools — APIs, databases, calculators, retrievers, etc.
3. Chain these steps together in a logical workflow or graph.

Think of it as the “middleware” that connects:

- User inputs
- AI reasoning
- APIs, databases, and validation systems
- Outputs or actions

LangChain (with LangGraph) acts as the orchestrator that manages the workflow:
→ Draft → Validate → Revise → Approve → Export

Core Concepts in Langchain

Concept	Description	Example in Our System
Prompt Templates	Predefined templates with variables for consistent prompting	“Generate the ‘Methods’ section for a Phase 3 oncology trial using {study_title} and {protocol_data}.”
Chains	Sequences of LLM calls and tool uses	Retrieval → Draft → Validation
Agents	LLMs that can decide which tools to use dynamically	The “Compliance Agent” that decides to run the rule checker or rerun retrieval
Tools	Functions or APIs the LLM can call	Validator, Retriever, Exporter
Retrievers	Components that fetch relevant documents or data	Vector DB fetching relevant ICH or GCP guidelines
Memory	Context persistence across turns	Keeps track of prior sections drafted or reviewer comments
LangGraph	State machine extension for complex workflows	Loops between “Draft” and “Validate” until “Compliant” state is achieved

1. Retrieval Tool

Multiple **LangChain tools** (MCP-style) is needed, each mapped to the architecture.

Purpose: Retrieve relevant guidance, regulations, SOPs, or previous documents.
Framework: `langchain.retrievers`
Backend: **Vector database (e.g., Chroma, Pinecone, FAISS, or pgvector)**

You can connect this retriever to:

- **ICH E6(R3)**
- **21 CFR Part 11**
- **Internal SOPs**
- **Previous protocol templates**

2. Compliance Validation Tool

Purpose: Automatically check generated content against regulatory rules.
Framework: Custom LangChain **Tool** + rule engine (e.g., JSON schema or SpaCy-based rule matcher).

Rule sources:

- ICH, GxP, Part 11 checklists
- Sponsor-specific SOPs
- Terminology or section presence rules (“must include study rationale”)

3. Document Management / Export Tool

Purpose: Store versions, push final outputs to eTMF/CTMS.
Framework: Custom LangChain **Tool** wrapping file APIs or integration SDKs.

Utility tools

Purpose: Common helper tasks — calculation, translation, table formatting, glossary lookup, date normalization.

Examples:

- `calculator` (built-in LangChain tool)
- `python_tool` (for table calculations)
- `text_splitter` (for large doc processing)
- `doc_formatter` (convert markdown → docx/PDF)

Communication/Collaboration Tools

Purpose: Integrate with Slack, email, or GitHub for reviews.

Examples:

- `send_to_slack`: Post draft or compliance report for human review.
- `log_run`: Store results in audit DB or observability stack.

LLM Integration

LangChain connects directly to **Claude 3.5 Sonnet** (via Anthropic API) or through **Bedrock** if AWS-hosted.

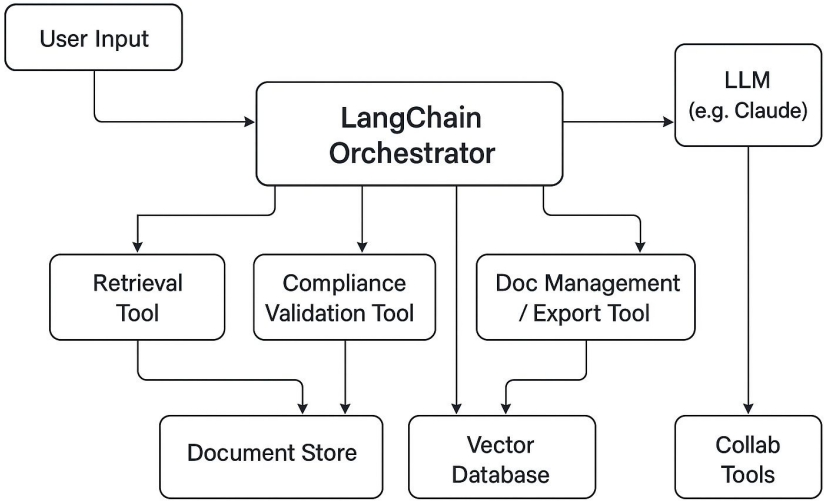
Used for:

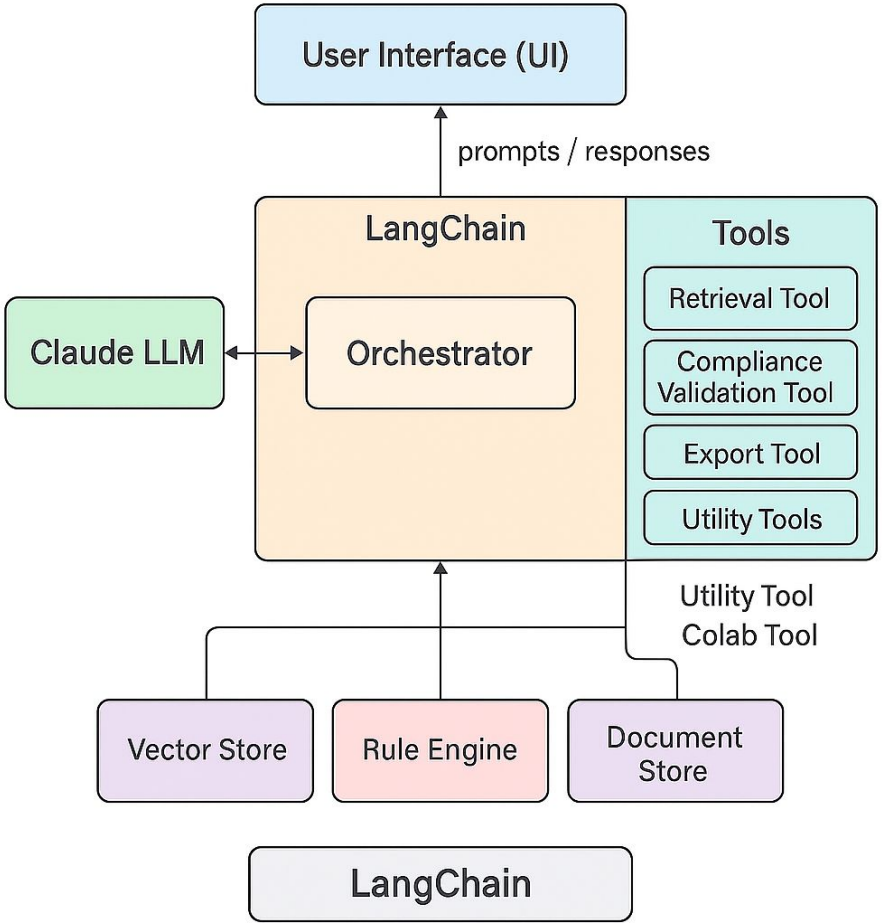
- Draft generation
- Revising based on feedback
- Compliance reasoning summaries

Supporting Components

Function	Tech Stack
Vector Store	Chroma / Pinecone / FAISS / pgvector
Rule Engine	JSON/YAML rules, Python DSL, or AWS Bedrock Guardrails
Audit Logging	LangSmith / custom SQL logs
Prompt Versioning	LangSmith / Weights & Biases
UI	React + FastAPI backend

Detailed Component Diagram





LangGraph Workflow

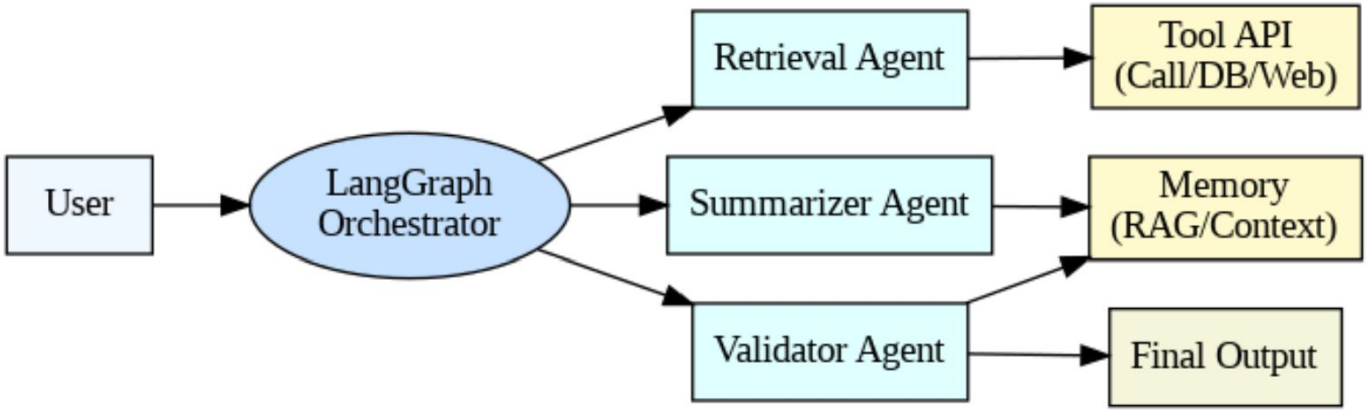
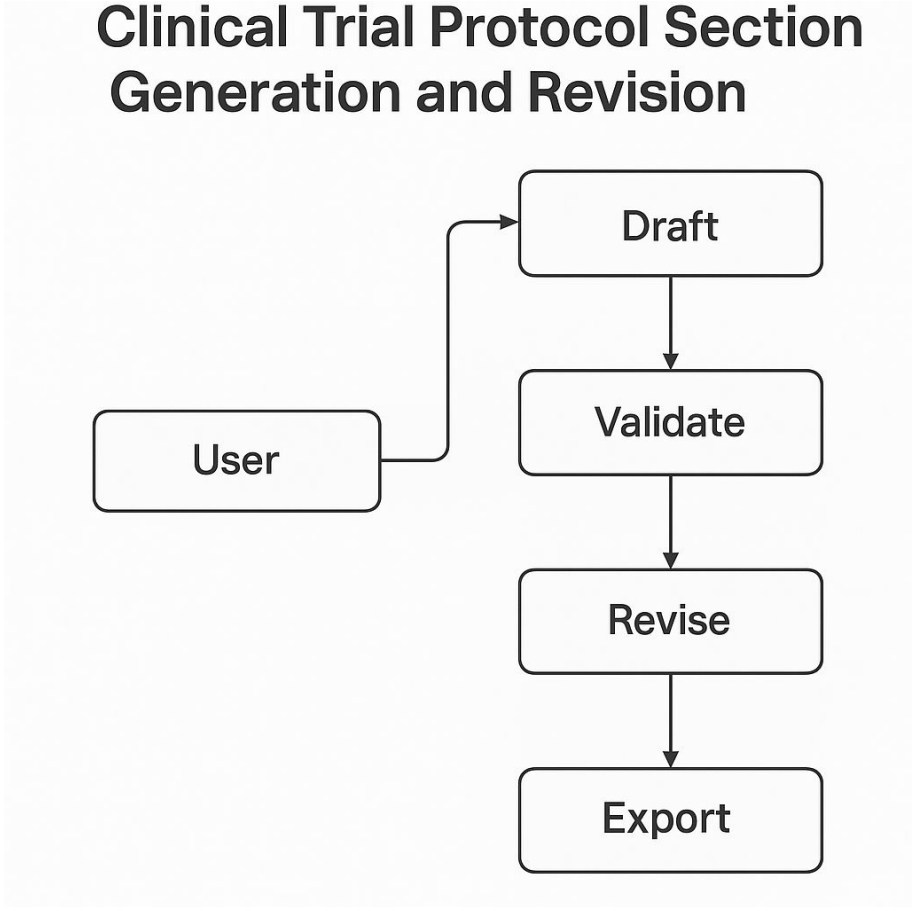


Figure: Workflow Architecture of a LangGraph-Orchestrated Multi-Agent LLM System

Claude 3.5 Sonnet



Tool Use

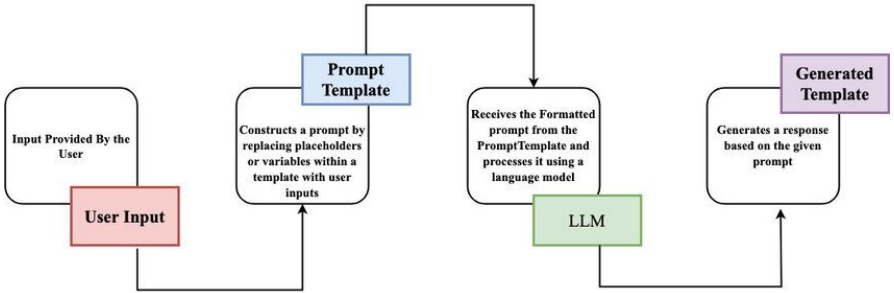
- Supports Model Context Protocol (MCP) for tool integration
- External tools: web search, GitHub, Slack, databases
- Dynamic invocation of functions, APIs, workflows

Workflow

- Step 1: User query → Check context needs
- Step 2: Optional tool call (search, DB, API)
- Step 3: Claude integrates results with reasoning
- Step 4: Respond with grounded, contextualized output

Langchain Workflow

- Provides structure for agentic workflows
- Enables chaining of tasks (draft → validate → revise → approve)
- Supports integration with tools: RAG, compliance checks, exporters
- Fits regulated workflows requiring audit trails



Tuning

- Fine-tuning via AWS Bedrock (e.g., Claude 3 Haiku)
- Prompt engineering for domain adaptation
- Parameter-efficient methods (LoRA, adapters)
- Combine tuning + retrieval for best results

Non-Parametric Continual Learning

- External memory integration without retraining parameters
- Retrieval-Augmented Generation (RAG) for continual learning
- Research frontier: HippoRAG 2 (associative memory)
- Benefits: avoids catastrophic forgetting, improves factuality

Reasoning and Tool Integration

- Strong multi-step reasoning with Constitutional AI
- Chain-of-thought style deliberation
- Tools extend reasoning: calculators, RAG, APIs
- Secure orchestration of multiple tools for complex tasks

Model Context Protocol

- Open standard by Anthropic for connecting models to tools
- Servers expose Tools, Resources, Prompts
- Claude Desktop acts as MCP client
- Benefits: standardization, security, extensibility

Agent-to-Agent

- Claude can interact with other agents via shared protocols
- Agent collaboration: divide tasks, cross-check outputs
- Applications: customer support triage, research assistants
- Challenges: coordination, consistency, accountability

How LangChain/LangGraph + Claude work together

- LangChain handles tool orchestration and context management
  - The process of providing a LLM with the right information and tools at the right time to help it accomplish a task
- LangGraph defines the workflow as a state machine
- Claude generates, revises, and validates text at each step
- Together they support iterative compliance-driven drafting

Project Workflow

1. Draft: Claude generates a trial protocol section
2. Validate: Compliance tool checks regulatory requirements
3. Revise: Claude fixes issues flagged by validator
4. Finalize: Human-in-the-loop approves the draft
5. Export: Document saved to eTMF/CTMS system

Technical Objectives

1. Advance long-context reasoning

   Improve Claude’s ability to reliably process, recall, and reason over extremely long contexts (hundreds of thousands of tokens or more

2. Strengthen tool & protocol integration

   Deepen Claude’s ability to securely and effectively use external tools, via the Model Context Protocol (MCP), Retrieval-Augmented Generation (RAG), and structured APIs.

3. Enhance safety-aligned reasoning at scale

   Refine Claude’s step-by-step reasoning so that it is transparent, reliable, and aligned with human intent, even under adversarial or ambiguous prompts

Proposed Original Contributions

1. **Adaptive Hierarchical Memory Encoding (AHME)**

   Introduce a dynamic memory layer that splits long input into hierarchical levels (e.g., sentences → sections → documents).

2. **Secure Multi-Tool Orchestration with Intent Certificates (SMTO-IC)**

   Extend MCP with a lightweight **intent-certification layer**, where each tool call must carry a structured “reason-for-use” tag, signed by the model’s reasoning engine.

3. **Constitutional Reasoning Graphs (CRG)**

   Represent Claude’s internal ethical reasoning as a **transparent graph structure**, where each node corresponds to a principle (e.g., fairness, privacy, harm avoidance).

Literature References

Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI Feedback.
arXiv:2212.08073.
Foundational paper describing Claude's alignment method, where a "constitution" of principles guides reinforcement learning.

Askell, A., et al. (2021). A General Language Assistant as a Laboratory for Alignment.
arXiv:2112.00861.
Early Anthropic research framing LLM assistants as testbeds for alignment and safety research.

Anthropic (2023). Claude 2 Release Blog.
Anthropic Blog.
Official documentation on Claude's capabilities, limitations, and design choices.

Anthropic (2024). Claude 3 Family Announcement.
Anthropic Newsroom.
Overview of Claude 3 series (Haiku, Sonnet, Opus), benchmarks, and multimodal capabilities.

Chataut, Sandeep & Do, Tuyen & Aryal, Shiva & Khanal, Anup & Lushbough, Carol & Gnimpieba, Etienne & Shrestha Gurung, Bichar. (2024).
COMPARATIVE STUDY OF DOMAIN DRIVEN TERMS EXTRACTION USING LARGE LANGUAGE MODELS. 10.48550/arXiv.2404.02330.